7CS033 - NLP & Sentiment Analysis

Jordan Brennan

Faculty of Science & Engineering
University of Wolverhampton

What is Natural Language Processing?

Natural Language Processing (NLP) is a data science technique that allows computers to understand natural language, as a person does.

NLP combines **computational linguistics** with **machine learning techniques** to allow computers to comprehend both text and speech.

NLP has wide ranging applications in HCI, one of the biggest being **information retrieval and search engines**. NLP is also extremely useful for data analysis, enabling us to extract useful insights from unstructured text data. Such as **sentiment analysis** of customer reviews to inform marketing decisions.

NLP has also paved the way for modern generative AI methods, such as LLM impressive communication skills.

Data Cleaning

In nearly all NLP applications, data cleaning is needed. NLP offers unique challenges we do not see elsewhere as text data is often **unstructured**. Raw text data can contain information that can be confusing or offer no meaningful insights for the given task.

Example 1: Look at the following text data, what do you notice?

$$D_1 = \text{"I LOVED this book!!! <3 <3"}$$
$$D_2 = \text{"I loved this book"}$$

There are many different techniques for cleaning text data. We will discuss some of the most common methods today.

Data Cleaning - Lowercasing

Lowercasing is a technique used to normalise text. It involves converting all text to lowercase. For example:

$$\text{Cat, cat, CAT} \rightarrow \text{cat}$$

This is an important part of the process as by lowercasing text we remove duplicates from our vocabulary.

In the above example "Cat", "cat" and "CAT" would all be considerered as unique words/tokens. This can be problematic as we likely do not want our models to treat these words differently.

Data Cleaning - Punctuation Removal

Punctuation removal is the process of removing all punctuation from our text. For example:

"I loved this book!!!!" $\rightarrow$ "I loved this book"

Punctuation may or may not offer semantic meaning. Traditionally, punctuation was considered to carry little semantic meaning and is removed. Removing punctuation also decreases the size of the data and can help improve computational efficiency.

However, in the example above we can see that the "!!!" could imply something about the writers thoughts and feelings. Therefore, punctuation removal can be task dependent.

What about **emojis**?

Data Cleaning - Stopword Removal

Stopword removal is the process of removing words that occur frequently in the dataset. Common stopwords are words such as "the", "is", "and, "of" etc. For example, let us suppose we had the document D_1 = "The book was very good".

Then our vocabulary would be
$V = \{$"the", "book", "was", "very", "good"$\}$.

After removing the stopwords our vocabulary is reduced to
$V = \{$"book", "good"$\}$.

Therefore, removing stopwords helps us to reduce the size of our data. It also helps us focus on only informative words. We must be sure we apply this process appropriately as not all frequently occuring words should be considered stopwords. If the word "not" was common in a document, this does not mean we would remove it as "not" change the sentiment of a text.

Data Cleaning - Spelling Correction

- Where might we extract text for further analysis?

Data Cleaning - Spelling Correction

- Where might we extract text for further analysis?
- Text data extracted from customer reviews, blogs or social media have a chance of containing spelling mistakes.
- We want to correct these mistakes to ensure first that the intended sentiment of the writer is being expressed.
- Removing incorrect spellings also helps in that we reduce the size of our data by removing the spurious spellings.
- In turn this will help any models we train on the data improve their accuracy as they won't be learning meaningless patterns from misspelled words.
- Can you think of any problems with removing incorrect spellings?

Data Cleaning - Tokenization

Tokenization is a process that involves splitting text into meaningful units. Usually, tokenization refers to splitting text into tokens of words. There are other methods of tokenizing such as: sentence, subword or character tokenizers.

Given a document D_1 = "the book was very good". After tokenization we would have a set of tokens:

["book", "good", "the", "very", "was"]

Language matters!

Data Cleaning - Stemming

Stemming is the process of converting words into their root words using some set of rules irrespective of meaning. One popular method for stemming in English is the **Porter Stemmer**. The algorithm works by looking for common endings to words and removing them to reduce the words to their **stems**. For example:

$$\text{running} \rightarrow \text{run}$$
$$\text{happiness} \rightarrow \text{happi}$$
$$\text{happy} \rightarrow \text{happi}$$

Stemming reduces the vocabulary size. The words produced by stemming do not necessarily have to be words. This can make interpreting stems difficult. This process can also reduce two different words to the same stem which could mean information is lost. This algorithm is designed for English and cannot be used with other languages.

Data Cleaning - Lemmatization

Lemmatization is converting words to their root word using vocabulary mapping. Lemmatization is based on the meaning of words more than the structure of a word. For example:

$$\text{good} \rightarrow \text{good}$$
$$\text{better} \rightarrow \text{good}$$
$$\text{best} \rightarrow \text{good}$$

We can see that lemmatization converts synonyms into a single root word. It is argued that lemmatization is more robust than just stemming as it uses not just the words themselves but also knowledge about the meaning of words to create the lemmas.

Exploratory Data Analysis - Word Frequency

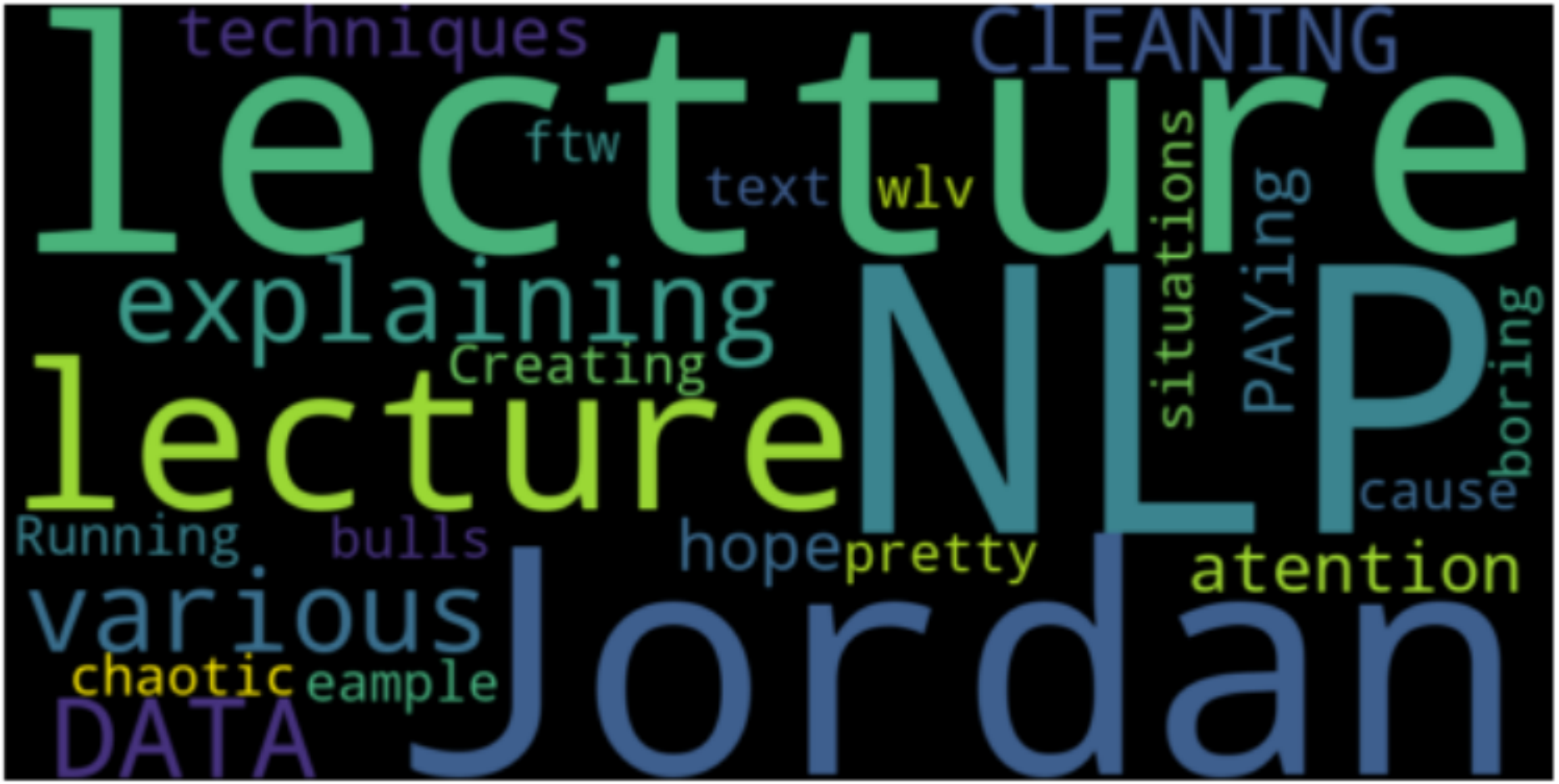
It can be useful to explore any dataset. This is especially true with text data since this type of data is often **unstructured** and can offer unique challenges that would not be present in structured data.

Counting the unique words in a data can give us an idea about our data's most frequent and least frequent terms.

Understanding this, we can make **informed decisions** to help in our future work. For example we may find that by removing the least frequent terms in our dataset we can train a more generalised model.

Exploratory Data Analysis - Word Clouds

A **word cloud** is a pictorial representation of the word frequency of a dataset. A word cloud is often easier to understand and gives a better idea about the textual data.



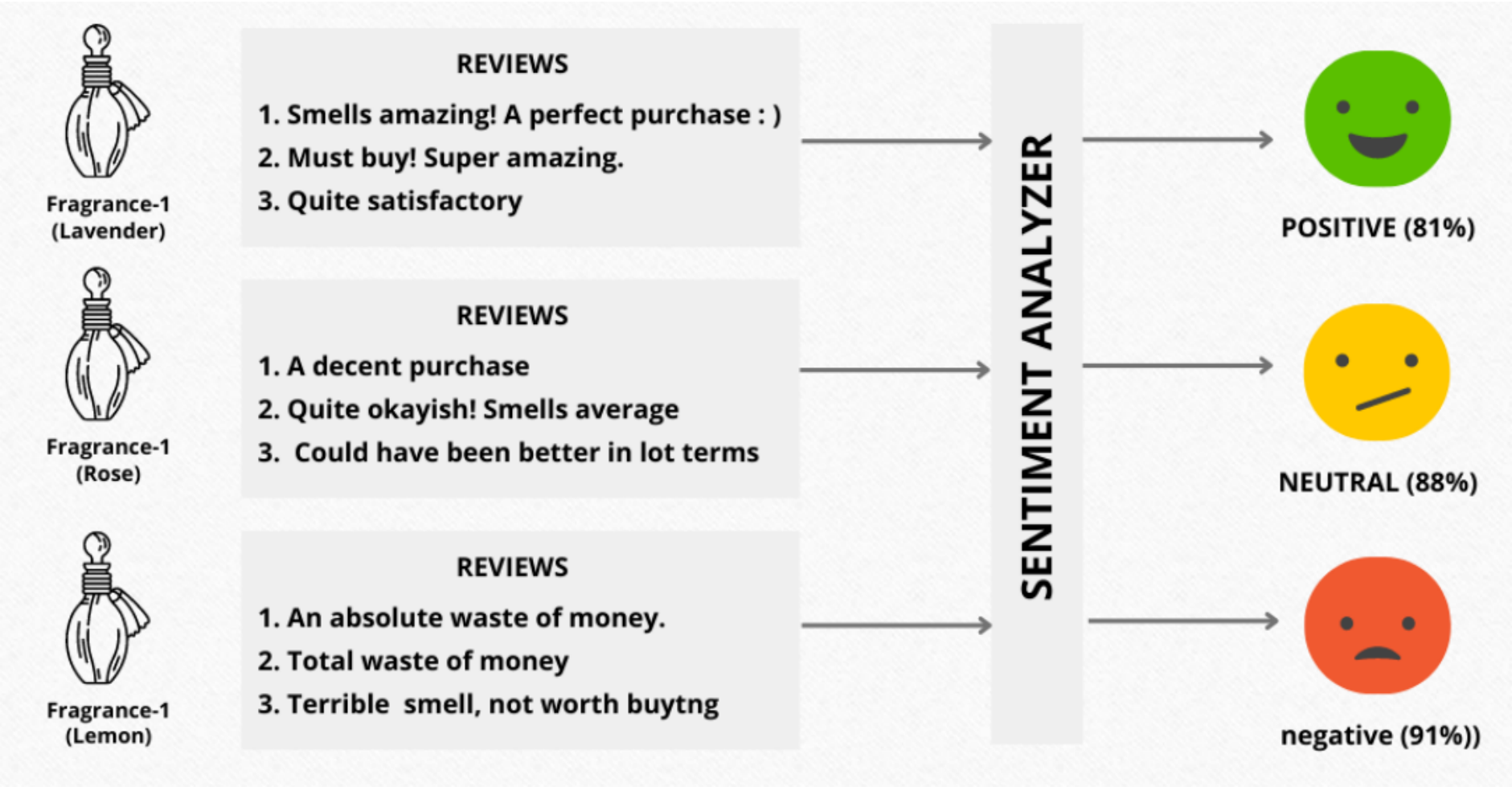
Sentiment Analysis

Sentiment analysis is a use case of Natural Language Processing and comes under the category of text classification.

Sentiment analysis is a technique that aims to detect the underlying sentient of a piece of text.

It is the process of classifying text as either: positive, negative or neutral. Machine learning techniques are used to evaluate a piece of text and determine the sentiment behind it.

Sentiment Analysis - Examples



Sentiment Analysis - Use Cases

- **Social Media Monitoring for Brand Management**: Brands can use sentiment analysis to gauge their Brands public perception. For example, a company can gather all the Tweets in which the company is mentioned or tagged and perform sentiment analysis to learn how the company is currently being perceived .
- **Product/Service Analysis**: Organisations can perform sentiment analysis on customer reviews to see how well a product or service is doing in the market and make future decisions accordingly.
- **Stock Price Prediction**: Predicting whether stocks of a company will go up or down is crucial for investors. If the news headlines pertaining to a particular organisation happen to have a positive sentiment, we would expect stock prices to increase, and vice-versa for negative sentiment.

Sentiment Analysis Steps

The general pipeline for sentiment analysis is as follows:

- **Text Preprocessing**: The text data is cleaned by removing irrelevant information such as special characters, punctuation and stopwords.
- **Tokenisation**: The text is divided into individual words or tokens to facilitate analysis.
- **Feature Extraction**: Relevant features are extracted fro the text such as words or parts of speech.
- **Labelling the Text**: In this step, we determine whether a text is positive, negative or neutral.
- **Sentiment Classification**: Machine learning algorithms are used to classify the labeled text data.
- **Evaluation**: The performance of the sentiment analysis model is assessed using evaluation metrics such as accuracy, precision, recall or F1 score.

Web Scraping

Web scraping is the process of automatically extracting data from a website by parsing the HTML of the webpage.

Every website has rules, do not assume that you are free to scrape any site. Each site will have its own terms and conditions, check the *robots.txt* file that each website has. ***example.com/robots.txt***.

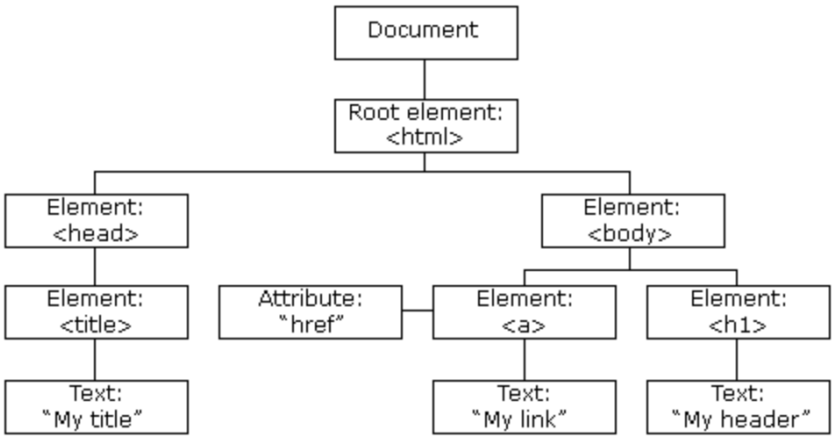
To scrape websites we must understand the structure of the site. Websites are structured text written in HTML:

```
<h1>Article Title</h1>
<p>This book was fantastic.</p>
```

Webpage Structure

Webpages are written as structured text using **HyperText Markup Language (HTML)**. Your browser interprets this structure to display the content. HTML uses tags to organise the information and describe the purpose of the content.

A webpage can be viewed as a tree of connected elements called **The Document Object Model**. In HTML DOM every element is a node.



We locate data by targeting elements in this structure. HTML elements often have attributes. Attributes can give us additional information about elements, class="review".

Basic Web Scraping Steps

The general pipeline for web scraping is as follows:

- **Request the Webpage**: Identify a webpage and send a request to the relevant URLs to get the HTML of the page. Libraries such as **Requests** can be used for this.
- **Parse the HTML**: Convert the raw HTML data into a searchable, structured format. **Beautiful Soup** is one of the most popular Python libraries for parsing HTML.
- **Extract Data**: Extract the relevant text data and store in an appropriate format such as a *.csv* or a dataframe.

Often we use web scraping **Application Programming Interfaces (API)**. These are specialised interfaces that interact with webpages to facilitate the extraction of data from websites. It is often much easier to use an API however they are often costly.

Lets Practice